



Universidad Nacional Autónoma De México
Facultad de Ingeniería
División Ingeniería Eléctrica

Semestre 2026-2 Grupo: 5
Semestre 2026-2

Estructura y Programación de Computadoras

Reporte técnico - Proyecto Final

Profesora: Ing. Ariel Adara Mercado Martinez.

Integrantes:

ESPEJEL ORNELAS IRVIN GIOVANNI
GONZÁLEZ MANCILLA JOSÉ LUIS (1)
RIOS CRUZ RICARDO
SALINAS RUIZ CRISTIAN DANIEL
VILLAVICENCIO ZÁRATE MIGUEL ÁNGEL

Fecha de Realización: 30 de Mayo del 2025
Fecha de Entrega: 11 de Junio del 2026

Introducción

Los ensambladores constituyen una herramienta fundamental en el desarrollo de software de bajo nivel, ya que permiten traducir instrucciones escritas en lenguaje ensamblador a código máquina ejecutable por el procesador. Comprender su funcionamiento interno resulta esencial para entender cómo se realiza la generación de programas ejecutables, el manejo de memoria y la interacción directa con la arquitectura del sistema.

En este proyecto se desarrolló una implementación simplificada de un ensamblador para la arquitectura IA-32 utilizando el lenguaje de programación C. El sistema fue diseñado de manera modular, distribuyendo responsabilidades entre los integrantes del equipo para cubrir las distintas etapas del proceso de ensamblado, incluyendo análisis léxico, análisis sintáctico, construcción de tablas de símbolos, generación de código máquina, soporte para direccionamiento mediante ModRM y SIB, generación de archivos objeto y enlazado básico.

Objetivo

Comprender internamente:

- Cómo funciona un ensamblador.
- Cómo se genera código máquina.
- Cómo funcionan los archivos objeto.
- Cómo funcionan las relocalaciones.
- Cómo funciona un linker.
- Cómo se construyen ejecutables.

Desarrollo

Integrante 1

El propósito del analizador Léxico (Lexer), es tomar el código fuente de un archivo **.asm** escrito para la arquitectura IA-32, descartar todo aquello que no aporta información sintáctica (como espacios en blanco, tabuladores y comentarios) y clasificar cadenas restantes en unidades mínimas con significados: Los tokens. La salida de este módulo es la materia prima (o lo primordial) que necesita el analizador en la siguiente etapa del compilador.

Tokenización

En esta etapa la prioridad es separar los componentes sintácticos útiles y eliminar el ruido visual (basura). Se recurre a **strtok** de la biblioteca estándar de C, con el siguiente conjunto de delimitadores definido explícitamente en el código:

```
strtok(linea_actual, " ,\n\r\t[]+*")
```



Estos delimitadores cubren espacios, comas, saltos de línea, tabuladores y los caracteres propios del ensamblador IA-32 como corchetes, signos de suma y asterisco. El resultado de todo esto son cadenas limpias (lexemas) listas para ser clasificadas.

Validación léxica:

Se integró la función **estandarizar_token** que convierte cada lexema a mayúsculas carácter a carácter antes de cualquier comparación. Esto nos garantiza que instrucciones escritas de distintas maneras (mOv, Mov, MOv) sean reconocidas sin errores de sensibilidad al caso.

Manejo de comentarios:

El Lexer detecta el carácter `<<;>>` mediante **“strchr”**. Al encontrarlo, escribe un terminador nulo en esa posición, truncando la cadena. El texto que sigue en esa línea se descarta por completo, incluso cuando el comentario no está separado por espacios del código inmediatamente anterior.

Tabla de búsqueda y clasificación de Tokens:

El núcleo de la clasificación es la función **clasificar_lexema**, apoyada en un arreglo estático de 27 entradas (DiccionarioToken tabla_busqueda[]) que cubre tres categorías de elementos léxicos

Categoría	Token resultante	Ejemplo reconocidos
Registros (8)	REG_EAX, ..., REG_ESP	EAX, EBX, ECX, EDX, ESI, EDI, EBP, ESP
Instrucciones (1)	INSTA_MOV, ..., INST_INT	MOV, PUSH, POP, LEA, ADD, SUB, INC, DEC, JMP, JE, JNE, NOP, INT
Directivas (6)	DIR_DB, ..., DIR_EXTERN	DB, DW, DD, SECTION, GLOBAL, EXTERN
Etiquetas	TOKEN_ETIQUETA	Cualquier lexema que termina en ':'



Números	TOKEN_INMEDIATO	Enteros positivos y negativos (ej. -450)
Identificadores	TOKEN_IDENTIFICADOR	Inician con letra, '_' o '.' (ej. .data, _var1)
Desconocido	TOKEN_DESCONOCIDO	Símbolos ilegales: @, ?, etc.

Manejo básico de errores

El compilador **NO** se detiene ante caracteres ajenos a la sintaxis esperada del ensamblador, como @ o ?. En su lugar, el Lexer lo que hara es aislar el símbolo, lo marca internamente como TOKEN_DESCONOCIDO y emite una advertencia en consola con el siguiente formato:

```
[LEXER] Abriendo: examples/test.asm
>> [ADVERTENCIA LEXICA] Simbolo no reconocido: '@SIMBOLO_ILEGAL'
>> [ADVERTENCIA LEXICA] Simbolo no reconocido: '?ETIQUETA_ROTA'
[LEXER] 28 tokens extraídos.
```

```
manxilla@Mxilla: /mnt/c/Users/josel/Desktop/2026-2/proyecto-final-team1-fixed/proyecto-final-team1-fixed$ ./ensamblador examples/test.asm
=== Ensamblador IA-32 - proyecto final ===

[LEXER] Abriendo: examples/test.asm
>> [ADVERTENCIA LEXICA] Simbolo no reconocido: '@SIMBOLO_ILEGAL'
>> [ADVERTENCIA LEXICA] Simbolo no reconocido: '?ETIQUETA_ROTA'
[LEXER] 28 tokens extraídos.

[BACKEND] Cambiando a seccion .DATA. LC restaurado en: 0
[BACKEND] Cambiando a seccion .TEXT. LC restaurado en: 0
Error de sintaxis: operando no reconocido '@SIMBOLO_ILEGAL'
Error de sintaxis: operando no reconocido '?ETIQUETA_ROTA'
Error de sintaxis: operando no reconocido 'ADD'
Error de sintaxis: operando no reconocido 'NOP'
[PASADA 1] Completada. 2 simbolos registrados. LC final: 21

[PASADA 2] Iniciando generacion de codigo maquina...
[ENCODER] Instruccion no soportada: ADD
Error de sintaxis: operando no reconocido '@SIMBOLO_ILEGAL'
[ENCODER] Instruccion no soportada: MOV
Error de sintaxis: operando no reconocido '?ETIQUETA_ROTA'
Error de sintaxis: operando no reconocido 'ADD'
Error de sintaxis: operando no reconocido 'NOP'
[ENCODER] Instruccion no soportada: INC
[PASADA 2] Completada. 10 bytes generados.

[OBJ] Archivo objeto generado: salida.obj

[MAIN] Archivo objeto generado: salida.obj

--- Tabla de simbolos ---
_START      addr=0x0000  sec=.TEXT  global=1  extern=0
ETIQUETA_VALIDA  addr=0x0012  sec=.TEXT  global=0  extern=0

--- Codigo maquina (10 bytes) ---
B8 00 00 00 00 00 E9 00 00 00 00

=== Ensamblado finalizado ===
```



El objetivo principal de este módulo es recibir el flujo de componentes léxicos (tokens) generados por la fase de análisis léxico y construir una representación interna estructurada (Árbol de Sintaxis Abstracta o AST). Esta abstracción aísla la complejidad del texto plano y proporciona al Backend y al Encoder estructuras de datos tipadas y listas para la resolución de direcciones y la generación de código máquina IA-32.

Debido a las restricciones del proyecto, que prohíben el uso de generadores automáticos como yacc o bison, el analizador sintáctico fue implementado de manera manual utilizando la técnica de descenso recursivo en lenguaje C. Los archivos correspondientes a este módulo se encuentran estructurados en `include/parser.h` y `src/parser.c`.

Para modelar la gramática del subconjunto de instrucciones de IA-32, se diseñó un conjunto de structs que capturan el estado y los parámetros de cada línea de código ensamblador. El diseño priorizó la eficiencia en memoria y el fácil acceso para las fases posteriores del ensamblador:

- Estructura Operand: Es el núcleo computacional del parser. Se diseñó para soportar los distintos modos de direccionamiento requeridos. Para resolver la complejidad del direccionamiento de memoria, la estructura extrae y aísla los componentes del byte SIB (Scale, Index, Base) y el desplazamiento (displacement). Mediante banderas booleanas (`has_base`, `has_index`) y campos numéricos previamente procesados, el Encoder queda eximido de realizar análisis de cadenas.
- Estructura Instruction: Agrupa el mnemónico de la operación (ej. MOV, ADD) y aloja hasta dos estructuras Operand, llevando también un contador del número de operandos detectados para facilitar la validación semántica posterior.
- Estructura Directive: Un contenedor simplificado para procesar directivas del ensamblador (como SECTION .text o DB), almacenando el nombre de la directiva y un arreglo de argumentos en formato de cadena.

La lógica de parseo se implementó mediante funciones de evaluación mutuamente excluyentes (`parse_instruction`, `parse_directive`, `parse_operand`), las cuales reciben un puntero doble a la estructura de tokens (Token). Esto permite consumir el flujo de tokens secuencialmente, desplazando el puntero en memoria conforme se validan las reglas gramaticales de IA-32.

El procesamiento de operandos de memoria (`parse_memory_operand`) fue el segmento de mayor complejidad algorítmica. La función exige un token de apertura de corchete (`[`) e itera sobre los tokens internos hasta encontrar el cierre correspondiente (`]`). Durante este ciclo, el algoritmo clasifica los identificadores como registros base o índice, detecta los multiplicadores para establecer la escala permitida (1, 2, 4 u 8) y acumula las constantes numéricas para calcular el desplazamiento estático total.

El módulo incorpora rutinas de validación en tiempo de parseo para garantizar la correctitud del código ingresado. Por ejemplo, la función `parse_instruction` exige estrictamente la presencia del token `TOK_COMMA` (coma) cuando se detecta más de un operando antes del token de salto de línea (`TOK_NEWLINE`). Cualquier desviación de la gramática esperada aborta la ejecución con un error de sintaxis claro, protegiendo las fases de enlace y generación binaria de fallos de segmentación o códigos objeto corruptos.

Integrante 3

Diseñamos e integramos un componente modular en **C** que se encarga del núcleo lógico del ensamblador, conectando el análisis sintáctico (Frontend) con la generación de bytes (Backend).

Las funcionalidades incluyen:

- **Tabla de Símbolos Dinámica:** Funciones de búsqueda (`symbol lookup`) e inserción (`symbol insert`) para registrar etiquetas y variables asociadas a su dirección de memoria (`location counter`).
- **Manejo de Errores Semánticos:** Validación nativa para detectar e interrumpir el programa si el usuario intenta redefinir una etiqueta (duplicados).
- **Mapeo Exhaustivo de Secciones:** Control segmentado de memoria para las secciones obligatorias `.text`, `.data` y `.bss` mediante la función `cambiar de sección`, impidiendo que los contadores de variables interfieran con el código ejecutable.
- **Resolución de Referencias Adelantadas (Fixups):** Una tabla encargada de almacenar los saltos a etiquetas que aún no han sido declaradas en la primera pasada.
- **Corrección de Arquitectura IA-32:** Se ajustó la función de resolución (`fixup resolve all`) para calcular **desplazamientos relativos** en lugar de direcciones absolutas cuando se opera en la sección `.text`, respetando las especificaciones físicas de la arquitectura objetivo.

En la arquitectura IA-32, las instrucciones escritas en lenguaje ensamblador deben convertirse a código máquina para que puedan ser ejecutadas por el procesador. Este proceso se conoce como **codificación de instrucciones** (instruction encoding). Este módulo implementa un codificador básico capaz de transformar una representación interna de instrucciones ensamblador en sus correspondientes secuencias de bytes IA-32. Para ello se utilizan los componentes fundamentales del formato de instrucción de Intel, incluyendo:

- Opcode.
- ModRM.
- SIB.
- Displacement.
- Immediate.

El módulo está compuesto por dos archivos:

encoder.h: Define la interfaz pública del codificador. El cual recibe una estructura Instruction con la instrucción ya analizada y un buffer donde se almacenarán los bytes generados. Y entonces, retorna la cantidad de bytes producidos.

encoder.c: Contiene la implementación completa del proceso de codificación.

Además de la función principal, incluye varias funciones auxiliares:

Función	Descripción
reg_code()	Convierte registros IA-32 a sus códigos binarios
write_imm32()	Escribe inmediatos de 32 bits
make_modrm()	Construye un byte ModRM
make_sib()	Construye un byte SIB
write_disp32()	Escribe desplazamientos de 32 bits

Por otro lado, la función: `static int reg_code(const char* reg);`

Convierte nombres de registros IA-32 a los valores utilizados por el procesador. Estos códigos son utilizados para construir campos ModRM y opcodes dependientes del registro.

Por su parte, los operandos inmediatos se almacenan mediante: `write_imm32()`



La función divide un entero de 32 bits en cuatro bytes consecutivos siguiendo el formato Little Endian, utilizado por IA-32.

Ejemplo:

MOV EAX, 12345678h → Genera: 78 56 34 12

Además, muchas instrucciones IA-32 requieren un byte adicional llamado ModRM.

Su estructura es: | Mod | Reg | R/M | → 2 bits 3 bits 3 bits

La función make_modrm(mod, reg, rm) lo construye mediante:

(mod << 6) | (reg << 3) | rm

Ejemplo

MOV EAX, EBX → Opcode: 89 → ModRM: 11 011 000 → Resultado: 89 D8

Para direccionamientos más complejos IA-32 utiliza el byte **SIB** (Scale Index Base).

Su formato es: | Scale | Index | Base |

La función make_sib() permite construir dicho byte.

Los desplazamientos se utilizan para acceder a memoria.

Ejemplo:

MOV EAX, [EBX+4] → El valor: 4 es el displacement.

La función write_disp32() almacena este valor en formato Little Endian.

Instrucciones de control		Manipulación de pila	
Instrucción	Opcode	Instrucción	Codificación
NOP	90	PUSH reg	50 + reg
RET	C3	POP reg	58 + reg

Transferencia de datos

MOV reg, imm32

Formato: MOV EAX, 10 → Codificación: B8+r id → Ejemplo: B8 0A 00 00 00

MOV reg, reg

Formato: MOV EAX, EBX → Codificación: 89 /r → Ejemplo: 89 D8

MOV reg, memoria

Formato: MOV EAX, [EBX+4] → Opcode: 8B → Utiliza: ModRM y Displacement



MOV memoria, reg

Formato: MOV [EBX+4], EAX → Opcode: 89 → Utiliza: ModRM y Displacement

Operaciones aritméticas

ADD → ADD EAX, EBX → Opcode: 01

SUB → SUB EAX, EBX → Opcode: 29

Comparación

CMP → CMP EAX, EBX → Opcode: 39

Saltos

JMP → JMP etiqueta → Opcode: E9

CALL → CALL funcion → Opcode: E8

JE → JE etiqueta → Opcode: 0F 84

JNE → JNE etiqueta → Opcode: 0F 85

Integrante 5

El objetivo de este módulo fue implementar los componentes necesarios para almacenar el resultado del ensamblado en archivos objeto reutilizables y desarrollar un enlazador (linker) simplificado capaz de combinar múltiples módulos en un único binario final. Además, se incorporaron herramientas de depuración para inspeccionar la estructura interna de los archivos objeto generados.

Se implementó el módulo `object_file.c`, encargado de generar los archivos objeto a partir de la información producida por el ensamblador.

Este módulo recibe:

- El código máquina generado durante la segunda pasada.
- La tabla de símbolos construida por el backend.
- La tabla de fixups utilizada para referencias adelantadas.

La función principal del módulo es `write_object_file()`, la cual serializa toda la información del ensamblador y la almacena en disco siguiendo el formato definido para el proyecto.

También se desarrolló la funcionalidad necesaria para leer archivos objeto previamente generados. Esta capacidad es utilizada por el linker para recuperar la información de múltiples módulos antes de realizar el proceso de enlace.

El sistema utiliza la información de la tabla de fixups generada por el ensamblador para construir las entradas de relocación del archivo objeto.

Las relocalaciones permiten representar referencias a símbolos cuya dirección definitiva aún no se conoce durante el ensamblado. Estas referencias son almacenadas temporalmente y posteriormente resueltas por el linker cuando se conoce la ubicación final de cada módulo dentro del binario resultante. Este mecanismo es fundamental para soportar referencias adelantadas y llamadas entre módulos independientes.

Para soportar programas compuestos por múltiples módulos, se implementó el manejo de símbolos externos mediante las directivas GLOBAL y EXTERN.

Los símbolos marcados como globales pueden ser exportados para ser utilizados por otros módulos. Por otro lado, los símbolos externos representan referencias a elementos definidos en archivos distintos.

Durante el proceso de enlace, el linker construye una tabla global de símbolos y utiliza dicha información para localizar las definiciones correspondientes a cada símbolo externo encontrado.

Se desarrolló el módulo linker.c, cuya responsabilidad principal es combinar múltiples archivos objeto y generar un único binario final.

Resultados

Como resultado del proyecto se logró implementar una estructura funcional capaz de procesar instrucciones escritas en lenguaje ensamblador para IA-32 y transformarlas en su representación correspondiente en código máquina.

El sistema desarrollado permitió identificar instrucciones, registros, operandos y directivas básicas, además de generar la codificación de instrucciones mediante el uso de opcodes, bytes ModRM y SIB cuando fueron requeridos por los modos de direccionamiento utilizados. Asimismo, se implementó el manejo de desplazamientos (displacements) e inmediatos (immediates), permitiendo la construcción correcta de la secuencia de bytes correspondiente a cada instrucción soportada.

Durante las pruebas realizadas se verificó el funcionamiento de los módulos desarrollados por cada integrante, comprobando la correcta integración entre el análisis léxico, el análisis sintáctico, la generación de tablas de símbolos y la codificación de instrucciones.

Problemas encontrados

Durante el desarrollo del proyecto se presentaron diversas dificultades técnicas relacionadas con la interpretación de la arquitectura IA-32 y la integración de los distintos módulos desarrollados por el equipo.



Uno de los principales retos fue comprender la estructura de codificación de instrucciones especialmente en lo referente al uso de los bytes ModRM y SIB, ya que estos dependen directamente de los modos de direccionamiento empleados por cada instrucción. También surgieron problemas relacionados con la coordinación entre las representaciones internas generadas por el parser y los datos requeridos por el módulo encargado de la generación de código máquina.

Adicionalmente, se detectaron errores durante las pruebas iniciales debido a inconsistencias en el cálculo de desplazamientos, manejo de operandos y validación de formatos de instrucciones. Agregando que algunos archivos no se comunicaban correctamente, lo que impedía que siquiera se pudiese ejecutar bien.

Conclusiones

El desarrollo de este proyecto permitió comprender de manera práctica el funcionamiento interno de un ensamblador y los procesos involucrados en la traducción de instrucciones ensamblador a código máquina.

A través de la implementación de los distintos módulos fue posible identificar la importancia de cada etapa del proceso de ensamblado, desde el análisis léxico y sintáctico hasta la generación final de bytes ejecutables. Asimismo, se reforzaron conceptos relacionados con tablas de símbolos, referencias adelantadas, modos de direccionamiento y codificación de instrucciones para la arquitectura IA-32.